



Certified Penetration Testing Professional

CloudGoat Cheat Sheet

CloudGoat

Syntax	
<code>./cloudgoat.py [command] [sub-command] [--arg-name] [arg-value]</code>	
Options	
<code>create</code> Syntax: <code>create [scenario-name]</code>	Deploys a scenario to the AWS account you choose
<code>list</code> Syntax: <code>list</code> <code><scenario> all deployed undeployed</code>	Shows some information about all, undeployed, or deployed scenarios
<code>destroy</code> Syntax: <code>destroy <scenario> all [--profile <name>]</code>	Shuts down and deletes a [scenario-name]'s cloud resources
<code>config</code>	Allows to manage various aspects of your CloudGoat installation
<code>config whitelist</code>	Creates the whitelist file with the result
<code>config profile</code>	To instruct CloudGoat to use an AWS profile by name using this command
<code>config argcomplete</code>	Help for setting up argcomplete, enabling tab completion for CloudGoat
<code>help</code>	Provides contextual help about commands
<code>./cloudgoat.py [scenario-name]</code> <code>help</code>	Used to print to the console a summary of the scenario
<code>./kill.sh</code>	To destroy the CloudGoat environment

CloudGoat Commands

Commands	Description
<code>./cloudgoat.py create iam_privesc_by_rollback</code>	Restore full admin privileges
<code>./cloudgoat.py create cloud_breach_s3</code>	Acquire instance profile keys and use them to discover, access, and exfiltrate sensitive data
<code>./cloudgoat.py create iam_privesc_by_attachment</code>	Delete the EC2 instance "cg-super-critical-security-server."
<code>./cloudgoat.py create ec2_ssrf</code>	Invoke the "cg-lambda-[CloudGoat ID]" Lambda function
<code>./cloudgoat.py create rce_web_app</code>	Find a secret stored in the RDS database
<code>./cloudgoat.py create codebuild_secrets</code>	A pair of secret strings stored in a secure RDS database
<code>aws configure --profile cloudgoat</code>	Configure the AWS CLI to use the keys
<code>./cloudgoat.py config whitelist -auto</code>	Configure the IP whitelist
<code>./cloudgoat.py create rce_web_app --profile cloudgoat</code>	Create the CloudGoat "rce_web_app" scenario
<code>aws ec2 describe-instances --profile <user></code>	Check running instances
<code>aws ec2 stop-instances --instance-id <Instance Id> --profile <User></code>	Stop instance
<code>aws iam list-attached-role-policies --role-name ec2_role --profile <User></code>	Check policy
<code>aws iam get-policy --policy-arn arn:aws:iam::<arn>:policy:/ec2_ip_policy --profile <user></code>	The current version of the ec2_ip_policy
<code>aws iam get-policy --policy-arn arn:aws:iam::<arn>:policy/ec2_ip_policy --version-id v1 --profile <user></code>	Policy permissions

<pre>aws iam create-policy-version -- policy-arn arn:aws:iam::<arn>:policy/ec2_ip_ policy -policy-document file://var/www.html/escalated_pol icy.json --set-as-default</pre>	Create a new version of ec2_ip_policy and set it as a default one
<pre>aws s3 ls s3://<bucket> -- recursive --profile <User></pre>	Find which buckets we can list
<pre>aws elbv2 describe-load-balancers --profile lara</pre>	Check load balancers, we may have access
<pre>aws ec2 describe-security-groups --profile <User></pre>	Check security groups